

Exercise 5: An MCP Server and a Subagent

Session: 5 — MCP Servers & External Tools

Duration: 30 minutes, plus a 5-minute closing round

Project: The same BookStore API.

Goal

In this exercise you:

1. Connect an MCP server that lets Claude read the bookstore database. You ask one database question before and after, and compare the answers.
2. Create a subagent that checks the code for security problems. You write two lines: when Claude uses it, and which tools it may use.

Before you start

Titles in *italics* are slide titles from this session.

Where to work. Open a terminal in the `bookstore` folder, the one that contains `package.json`. Start Claude with `claude`. You stay in this folder for the whole exercise.

Chat or terminal. Everything you type goes to Claude, in the chat, except the `claude mcp` commands. Those go to the terminal, after you leave Claude with `/exit`.

Fresh session. Type `/exit`, then start `claude` again. Do not use `/clear`. Claude reads `.mcp.json` and the agent files only when it starts.

Training mode. The project's `CLAUDE.md` tells Claude to teach instead of answer: before it explains or fixes something, it asks you one question. It also tells Claude not to open `store.db` itself. Stuck, or out of time? Say "*just tell me*". Claude then answers directly.

The experiment tag. Prompts that must start the work without a question begin with this text:

[Exercise 5 experiment – execute directly, no leading questions.]

The tag switches training mode off for that one prompt. Copy those prompts exactly as printed, tag included.

Never edit `CLAUDE.md`. It holds training mode for sessions 6 to 8. Everything you make in this exercise goes in new files. The slide says: commit `.mcp.json`. In this course repo, git ignores it. Your copy stays on your machine.

Two files that came with the repo. The folder `mcp-sqlite`, inside `bookstore`, holds the MCP server: one Python file, `server.py`. The file `exercises/starters/security-auditor.md` is a starter file: a file you finish yourself, in task 4.

The database. The app creates `store.db` in this folder the first time it runs. Look for the file in your editor. Not there? Run `bun run start`, wait for `listening on :8080`, and stop it with Ctrl+C.

Notes. Some steps ask you to write something down. Use paper or a text file, not the chat. The closing round uses your notes.

Minute 14. Fourteen minutes after the start, the trainer says "task 4" out loud. Start task 4 then, even if task 3 is not finished.

Tasks

1. Ask without the MCP server (4 min)

This task produces a `permissions.deny` rule in `.claude/settings.json`, a prediction and three notes.

1. **Add a deny rule** (1 min). Start `claude` and paste:

```
[Exercise 5 experiment – execute directly, no leading questions.] Add the rule
Bash(sqlite3 *) to the deny list under permissions in .claude/settings.json. Keep
everything else in that file. Create the file if it does not exist.
```

Your hook from session 4 stays in the file.

2. **Write a prediction** (1 min). You will ask three things: the number of books, the author with the most books, and a SQL query. Write one sentence: which of the three can Claude get right from the source code alone, and which will be a guess?

3. **Ask** (2 min). Paste:

[Exercise 5 experiment – execute directly, no leading questions.] How many books are in the bookstore database? Which author has the most books? Write a SQL query that returns all books with their author name and average rating, sorted by rating, highest first.

Write down three notes: the book count, the author, and how sure Claude says it is (or "did not say"). Does Claude say that its answers come from the code, not from the database?

Why a deny rule. `CLAUDE.md` asks Claude not to open `store.db`, but Claude can still decide to do it. With a `permissions.deny` rule, Claude Code blocks the command before it runs (*MCP Permissions and Security*).

Done when: your prediction and three notes are written, and Claude did not open `store.db`.

2. Register the MCP server and approve it (6 min)

This task produces `.mcp.json` and a connected server.

The server has two tools: `get_table_definitions` and `execute_query` (*The /mcp Command*). It only runs `SELECT` queries. You start it with `uv`, which you installed in the preparation.

1. **Leave Claude** (1 min). Type `/exit`. The next command goes to the terminal, not to Claude.
2. **Register the server** (1 min). Run this in the terminal, on one line:

```
claude mcp add --scope project sqlite-bookstore -- uv run mcp-sqlite/server.py store.db
```

The part after `--` is the command that starts the server (*Configuring an MCP Server*). `--scope project` writes it to `.mcp.json` in this folder (*Under the Hood: .mcp.json*). Open that file: it holds the command you just typed. If `uv` is missing or fails: run `claude mcp remove sqlite-bookstore`, then the command above again with `python3` (on Windows: `python`) in place of `uv run`. The server needs only Python, no packages.

3. **Restart and approve** (2 min). Start `claude`. Claude Code asks whether it may use the server from `.mcp.json`. Answer yes: this is the approval. A project file can start any program on your machine. So Claude Code asks you once, per project. It

saves your answer in `.claude/settings.local.json`. Did you answer no? Type `/exit`, run `claude mcp reset-project-choices` in the terminal, and start `claude` again.

4. **Check** (1 min). Type `/mcp . sqlite-bookstore` shows as connected, with both tools. Not connected? Type `/exit` and run `claude mcp list` in the terminal: it shows the error. The usual causes: `store.db` is missing (see *Before you start*), or the command in `.mcp.json` has a typing error.

Done when: `/mcp` shows `sqlite-bookstore` connected, with two tools.

3. Ask again, with the MCP server (4 min)

This task produces three more notes.

1. **Same question** (2 min). Paste the prompt from task 1, step 3 again, tag included. Watch the tool calls: `get_table_definitions` first, then `execute_query`. The names appear as `mcp__sqlite-bookstore__execute_query` (*The /mcp Command*).
2. **Compare** (1 min). Write down the same three notes. Which answers changed? Was your prediction right?
3. **Ask one thing the code cannot tell** (1 min). Paste, and note the title Claude finds:

[Exercise 5 experiment – execute directly, no leading questions.] Which book has the highest average rating? Show its title, its author and the text of its three best reviews.

Why the two rounds differ. Without the server, Claude reads the code that fills the database at first start. The column names come from that code, so they can be right. The numbers are a guess. With the server, Claude makes two read-only tool calls, and the answer comes from the real data.

Done when: you have three notes per round, and you can say which answer in round one (task 1) was a guess.

4. Create the security-auditor subagent (9 min)

This task produces `.claude/agents/security-auditor.md` and the report of the security check, `docs/security-audit.md`.

A subagent is a markdown file. Its first lines, between two `---` lines, are the frontmatter (*Custom Subagents*). The body, the instructions for the check, is written for you in the starter file. You write two frontmatter lines: `description`, the trigger, and `tools`, what the agent may use.

1. **Let Claude create the file** (1 min). Paste:

```
[Exercise 5 experiment – execute directly, no leading questions.] Copy
exercises/starters/security-auditor.md to .claude/agents/security-auditor.md. Change
nothing in the content.
```

2. **Write the two lines** (3 min). Open `.claude/agents/security-auditor.md` in your editor and replace the two `(I write this)` values.

- `description`: Claude reads only this line when it decides whether to start the agent (*How Subagents Get Invoked*). Say what the agent checks. Say when Claude should use it: what a user might ask for, and which code change should start it. The weak version from the slide *Which Subagent Description Invokes Reliably?* is `Reviews code for security issues`. Write a better one, in your own words.
- `tools`: tool names, separated by commas, written as on the slide *Custom Subagents*. The agent must never change code. Give it the smallest set that still lets it find and read files.

Optional, if you have time: run `/mcp-coach 4` (4 is the task number). It reads your description, asks one question about it, and does not write the line for you.

3. **Test the trigger** (4 min). Type `/exit` and start `claude` again. Agents load at start. Ask, without naming the agent:

```
[Exercise 5 experiment – execute directly, no leading questions.] Do a security audit of
the bookstore API. Save the full report, exactly as it was produced, to docs/security-
audit.md.
```

Watch the tool calls. When the agent starts, Claude prints a line with the name `security-auditor`, and the check runs inside that agent. That means your description works. If you only see Claude's own `Read` and `Grep` calls, Claude did the check itself: your description did not start the agent. Improve it, `/exit`, start `claude` and ask again. After two tries without the agent, add *"use the security-auditor agent"* to the prompt. Then the report still gets written.

4. **Check the report** (1 min). Open `docs/security-audit.md`. Near the top it says `**Audited by**:` `security-auditor`. The starter file asks the agent to write that line. The agent cannot write files, so your own Claude session saved the report.

Done when: both `(I write this)` values are replaced, and `docs/security-audit.md` has the `Audited by` line.

Bonus (only if time remains)

Try to write through the server. Paste:

```
[Exercise 5 experiment – execute directly, no leading questions.] Set the rating of review
1 to 5 in the database.
```

`execute_query` runs `SELECT` only, and the server opens the database read-only. Claude should tell you that it cannot do this. The server can read and nothing more: that is least privilege from the slide *MCP Permissions and Security*.

Read the server. Open `mcp-sqlite/server.py`. You do not need to know Python. Find the two tool names, and find `mode=ro`, where the database is opened read-only. The whole server is one file of about 200 lines (*Building a Custom MCP Server*).

Run the plugin. The folder `bookstore-plugin`, inside `bookstore`, holds the skills from exercise 4, the hook, a finished `security-auditor` agent and the SQLite server, packaged as one plugin (*Plugins: Bundling It All Together*). Type `/exit`, then start Claude with `claude --plugin-dir bookstore-plugin`. Type `/help`: the skills now appear as `/bookstore-plugin:commit` and `/bookstore-plugin:changelog`, next to your own. Type `/mcp`: the plugin's server connected without an approval prompt. Its `README.md` explains what is different from your files.

Closing round (5 min)

First run `/verify-exercise 5` in Claude. It reads `.mcp.json`, calls the server, reads your agent file and the report, and asks you one question. Read its report while the trainer asks the room.

Have these answers ready:

- Which answer in round one was a guess, and how did round two show it?
- Did Claude call `get_table_definitions` before every query, or only once?
- Did the subagent start by itself? Which words in your `description` did that?

- One thing you would tell someone who skipped this session.